Experiment 9

Student Name: Aaditya Anand **UID: 23BCS10446**
Branch: CSE **Section/Group: KRG 3-A**
Semester: 6th **Date of Performance:21/04/2026**
Subject Name: System Design **Subject Code: 23CSH-314**

1. **Aim**: To design an online movie ticket booking system (similar to BookMyShow) that enables users to search movies, view show timings, select seats, and book tickets with high consistency and low latency.

2. **Objective**:

- To design a scalable and distributed architecture for an online movie ticket booking platform capable of handling high user traffic.
- To enable real-time seat availability updates using caching and synchronization mechanisms.
- To support end-to-end booking workflow, including movie search, seat selection, booking, and cancellation.
- To ensure strong consistency during seat booking to prevent double booking using locking mechanisms (e.g., Redis locks).
- To implement user booking history and notification system for tracking past bookings and sending confirmations via email/SMS.

3. **Tools Used:**

- **Frontend (ReactJS, HTML, CSS, JavaScript, WebSocket)**
  Developed a responsive user interface for movie browsing, seat selection, and real-time seat availability updates.
- **Backend (Node.js / Java / Spring Boot)**
  Built RESTful APIs for authentication, movie management, booking, payment processing, and user operations.
- **Database (PostgreSQL / MySQL)**
  Managed structured data including users, bookings, payments, and show information with strong consistency.
- **Redis**
  Used for caching frequently accessed data (movies, shows) and implementing seat locking to prevent double booking.

- **Message Queue (Kafka / RabbitMQ)**
  Enabled asynchronous communication for booking events, payment processing, and notification delivery.
- **WebSocket**
  Provided real-time updates for seat availability and booking status.
- **Deployment (AWS / GCP, Load Balancer, Docker)**
  Designed a scalable cloud-based architecture using containerization and load balancing for high availability.

4. System Requirements:

A. Functional Requirements

- Users should be able to register, login, and manage their profiles using secure authentication mechanisms.
- Users should be able to search movies based on city, date, language, and genre.
- Users should be able to view available theatres and show timings for selected movies.
- Users should be able to select seats and initiate the booking process.
- The system should temporarily lock selected seats during booking to prevent concurrent access.
- Users should be able to make payments using multiple payment methods (UPI, cards, wallets).
- The system should confirm booking only after successful payment processing.
- Users should receive booking confirmation notifications via email/SMS.
- Users should be able to view booking history and cancel tickets (if cancellation policy allows).

B. Non-Functional Requirements

1. Scalability

- The system should support **millions of concurrent users**, especially during peak times (e.g., new movie releases), using horizontal scaling and load balancing.

2. Consistency (CAP Theorem)

- The system should prioritize strong consistency over availability (CP system).
- Seat booking must ensure no double booking, using distributed locking mechanisms (e.g., Redis locks).

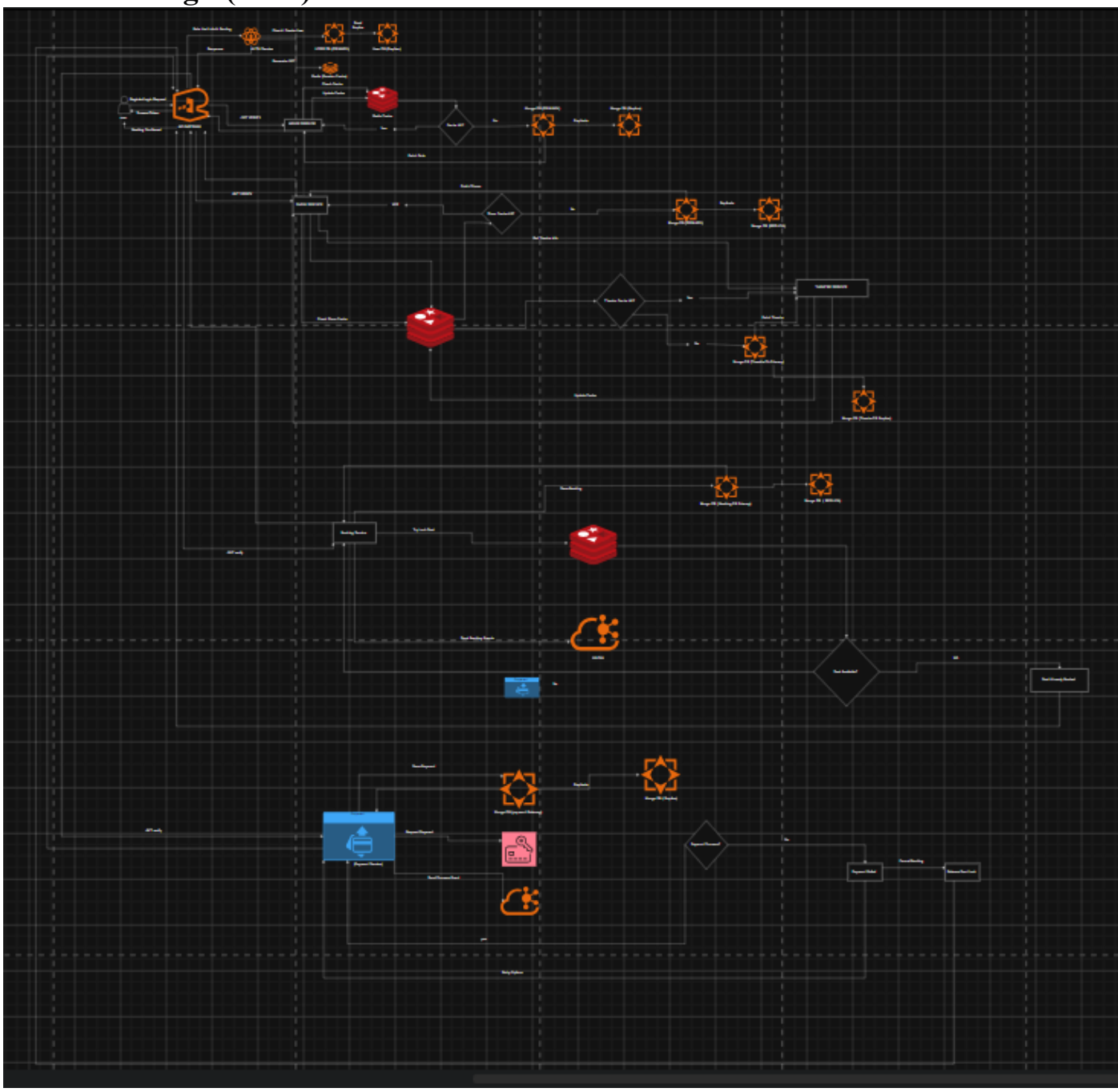
3. Latency (Performance)

- Movie search operations should respond within < 100 ms.
- Seat selection and booking operations should complete within 2–3 seconds.

4. Reliability

- The system should ensure no data loss during booking and payment operations.
- It should support fault tolerance and recovery mechanisms (e.g., retries, message queues).

5. Low Level Design (LLD):



6. Scalability Solution

- Use Redis for caching frequently accessed data (movies, shows) and implementing distributed seat locking to prevent double booking.
- Use Load Balancers to distribute incoming traffic evenly across multiple service instances for high availability.
- Adopt a Microservices Architecture (Auth, Movie, Show, Booking, Payment, Notification) to enable independent scaling of services.
- Implement Database Replication (read replicas) to handle high read traffic and Database Sharding to distribute large datasets.
- Use Asynchronous Message Queues (Kafka / RabbitMQ) for handling background tasks such as notifications and payment events.
- Optimize read-heavy operations using caching (Redis) to reduce database load and improve response time.
- Apply Horizontal Scaling (adding more servers/instances) to handle increasing user traffic efficiently.

7. Learning Outcomes

- Gained understanding of the architecture of real-time movie ticket booking systems.
- Learned the importance of strong consistency in seat booking to prevent double booking issues.
- Acquired knowledge of caching mechanisms and Redis-based distributed locking for performance and concurrency control.
- Understood the complete booking lifecycle (select → lock → pay → confirm) in a real-world system.
- Developed the ability to design scalable and low-latency distributed systems using microservices.
- Understood the trade-offs between consistency and availability (CAP theorem) in system design.